Curs 3a

Programare Orientată pe Obiecte în limbajul Java

Programare Orientată pe Obiecte

Distrugerea obiectelor

- **Obiectele care nu mai sunt referite vor fi distruse automat.**

Referinţele sunt distruse:

- **natural**
- **explicit, prin atribuirea valorii null.**

```
class Test {
    String a, c=new String();
    void init() {
        a = new String("aa");
        String b = new String("bb");
    }
    void stop() {
        a = null;
    }
}
```

Garbage Collector

- **Procesul responsabil cu eliberarea memoriei**

System.gc

”Sugerează” JVM să elibereze memoria

Finalizarea

- **Metoda finalize este apelată automat înainte de eliminarea unui obiect din memorie.**

finalize ≠ destructor

Declararea claselor

```
[public][abstract][final] class NumeClasa
  [extends NumeSuperclasa]
  [implements Interfata1 [, Interfata2 ...]]
  {
    // Corpul clasei
  }
```

- Moştenire simplă

```
class B extends A {...}
    // A este superclasa clasei B
    // B este o subclasa a clasei A
class C extends A,B // Incorect !
```

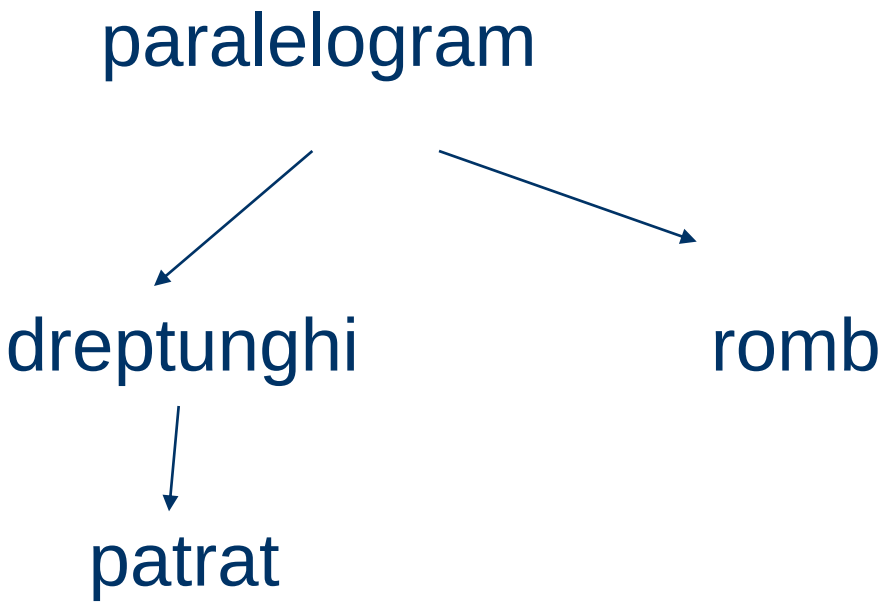
- Object este rădăcina ierarhiei claselor Java.

Moştenirea

- permite definirea şi crearea unor clase specializate plecând de la clase (generale) care sunt deja definite
- permite construirea unor clase noi, care păstrează caracteristicile şi comportarea, deci datele şi funcţiile membru (care nu sunt private), de la una sau multe clase definite anterior, numite clase de bază, fiind posibilă redefinirea sau adăugarea unor date şi funcţii noi.
- posibilitatea refolosirii lucrurilor care funcţionează
- ” Anumite obiecte sunt similare dar în acelaşi timp diferite”.
- *În Java există doar moştenire simplă,* o ierarhie de clase în care fiecare clasă derivată are o singură clasă de bază.

Principii POO: Moştenirea

Exemplu.



- Clasa *dreptunghi* este o clasă derivată (subclasă) a clasei *paralelogram*, iar clasa *paralelogram* este o clasă de bază (supraclasă) a clasei *dreptunghi*
- Astfel, o ierarhie de concepte conduce la o *ierarhie între clasele* care implementează conceptele ierarhice respective.

Clasa Object

Object este superclasa tuturor claselor.

class Exemplu {}

echivalent cu:

class Exemplu extends Object {}

Metode din clasa Object:

- **clone**
- **equals**
- **finalize**
- **toString …**

```
Exemplu obj = new Exemplu();
System.out.println("Obiect=" + obj);
//echivalent cu
System.out.println("Obiect=" + obj.toString());
```

Corpul unei clase

- **Variabile membre – date membru**
- **Constructori**
- **Metode membre**
- **Clase imbricate (interne)**

```
// C++
class A {
    void metoda1();
        int metoda2() { ... }
}
A::metoda1() { ... }

// Java
class A {
    void metoda1(){ ... }
    void metoda2(){ ... }
}
```

Constructorii unei clase

```
class NumeClasa {
    [modificatori] NumeClasa([argumente]) {
        // Constructor
    }
}
```

- Dacă pentru o clasă nu este definit niciun constructor, compilatorul iniţializează variabilele membru cu valorile lor implicite, în funcţie de tip, astfel:
  - Tipurile de date numerice cu 0
  - Tipul char cu caracterul vid (‘’)
  - Variabilele referinţă cu null

Apel explicit constructori - this

- **this apelează explicit un constructor al clasei.**

```
class Dreptunghi {
    double x, y, w, h;
    Dreptunghi(double x1, double y1, double w1, double
      h1) {
      // Implementam doar constructorul cel mai general
      x=x1; y=y1; w=w1; h=h1;
      System.out.println("Instantiere dreptunghi");
    }
    Dreptunghi(double w1, double h1) {
      this(0, 0, w1, h1);
      // Apelam constructorul cu 4 argumente
    }
    Dreptunghi() {
      this(0, 0);
      // Apelam constructorul cu 2 argumente
    }
}
```

Apel explicit constructori - super

super apelează explicit un constructor al superclasei!

```
class Patrat extends Dreptunghi {
     Patrat(double x, double y, double d) {
          super(x, y, d, d);
     }
     Patrat(double d){
       // super(d,d);    sau:
       this(0,0,d);
     }
}
```

Atenţie!

- **Apelul explicit al unui constructor nu poate apărea decât într-un alt constructor şi trebuie să fie prima instrucţiune din constructorul respectiv.**

this

this şi super: Sunt folosite în general pentru a rezolva conflicte de nume prin referirea explicită a unei variabile sau metode membre.

```
class A {
     int x;
     A() {
          this(0);
     }
     A(int x) {
        this.x = x;
     }
     void metoda() {
        x++; // nu este necesar this.x++; !!
     }
}
```

super

```
class B extends A {
    B(int x) {
        super(x);
    }
    B() {
        this(0); // nu scriem B(0) !!!!
    }
    void metoda() {  // supradefinire a metodei din B
        System.out.println("metoda clasei B" + x);
        // nu scriu super.x!! x exista in clasa B!
        super.metoda();
    }
}
```

Constructorul implicit

```
class Dreptunghi {
    double x, y, w, h;
    // Nici un constructor
}
class Cerc {
    double x, y, r;
    // Constructor cu 3 argumente
    Cerc (double x, double y, double r) { ... };
}
...
Dreptunghi d = new Dreptunghi();
// Corect (a fost generat constructorul implicit)
Dreptunghi d = new Dreptunghi(1,2,3,4);
// Incorect! Nu este generat implicit!

Cerc c;
c = new Cerc(); // Eroare la compilare !
c = new Cerc(0, 0, 100);  // Varianta corectă
```

Lanţul de apeluri al constructorilor

- Cazul în care o clasă de bază este extinsă (moştenită) de către o clasă copil.
- Constructorii nu se moştenesc!
- Constructorii pot fi invocaţi implicit sau explicit (*super*)

- Ori de câte ori este creat un obiect din clasa copil, este invocat mai întâi constructorul (fără parametri) al clasei părinte.

- Acesta este lanţul implicit de apeluri al constructorilor.

Modificatorii de acces

- ***Modificatorii de acces*** sunt cuvinte rezervate ce controlează accesul la membrii unei clase, sau chiar la o clasa.

Specificator	Clasa	Subcls*	Pachet	Oriunde
Private	X			
Implicit	X		X	
Protected	X	X	X	
Public	X	X	X	X

- dacă nu este specificat nici un modificator de acces, implicit nivelul de acces este la nivelul pachetului
- * subclasă din alt pachet

Declararea variabilelor

```
class NumeClasa {
    // Declararea variabilelor
    // Declararea metodelor
}
```

[modificatori] Tip numeVariabila [= valoare];

unde un modificator poate fi :

- **public, protected, private**
- **static, final, transient, volatile**

```
class Exemplu {
    double x;
    protected static int n;
    public String s = "abcd";
    private Point p = new Point(10, 10);
    final static long MAX = 100000L;
}
```

Final, transient, volatile

final:

```
class Test {
    final int MAX;
    Test() {
       MAX = 100; // Corect
       MAX = 200; // Eroare la compilare !
    }
}
```

transient:

- folosit la serializarea obiectelor, pentru a specifica ce variabile membre ale unui obiect nu participă la serializare.

volatile:

- folosit pentru a semnala compilatorului să nu execute anumite optimizări asupra membrilor unei clase.
- o facilitate avansată a limbajului Java.

Declararea metodelor

```
[modificatori] TipReturnat numeMetoda ([argumente])
    [throws TipExceptie1, TipExceptie2, ...]
{
    // Corpul metodei
}
```

unde un modificator poate fi :

- **public**, **protected**, **private**
- **static**, **abstract**, **final**, **native**, **synchronized**

```
class Student {
    ...
    final float calcMedie(float note[]) {
        ...
    }
}
class StudentInformatica extends Student {
    float calcMedie(float note[]) {
        return 10.00;
    }
}// Eroare la compilare !
```

Tipul returnat de o metodă

- return [valoare]
- void nu este implicit

```
public void afisareRezultat() {
        System.out.println("rezultat");
}
private void deseneaza(Shape s) {
        ...
        if …return;
        ….
}
```

- return trebuie să apară în toate situaţiile, atunci când am tip returnat:

```
double radical(double x) {
        if (x >= 0)
                return Math.sqrt(x);
        else {
                System.out.println("Argument negativ !");
                // Eroare la compilare
                // Lipseste return pe aceasta ramura!
        }
}
```

Argumentele metodelor

- Numele argumentelor primite trebuie să difere între ele şi nu trebuie să coincidă cu numele nici uneia din variabilele locale ale metodei.
- Pot să coincidă cu numele variabilelor membre ale clasei, caz în care diferenţierea dintre ele se va face prin intermediul variabile *this*.

```
class Cerc {
    int x, y, raza;
    public Cerc(int x, int y, int raza) {
        this.x = x;
        this.y = y;
        this.raza = raza;
    }
}
```

Trimiterea parametrilor (1)

TipReturnat metoda([Tip1 arg1, Tip2 arg2, ...])

- **Argumentele sunt trimise doar prin valoare (pass-by-value).**

```
void metoda(StringBuffer sir, int numar) {
    // StringBuffer este tip referinta
    // int este tip primitiv
    sir.append("abc");
    numar = 123;
}
...
StringBuffer s=new StringBuffer();
int n=0;
metoda(s, n);
System.out.println(s + ", " + n);
// s va fi "abc", dar n va fi 0
```

Clase wrapper

- Set de clase folosite la ambalarea (wrap) valorilor primitive într-un obiect.

byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

- Toate clasele wrapper sunt imuabile (immutable), de fiecare data când modificati valoarea unui obiect wrapper este creat un obiect nou

Obs: Vector nu Vector<Integer> etc

Conversii automate între tipuri – autoboxing si unboxing

- Înainte de versiunea 1.5

```
Integer obi = new Integer(1);
int i = obi.intValue();
Boolean obb = new Boolean(true);
boolean b = obb.booleanValue();
```

- Doar de la versiunea 1.5:

```
Boolean obb = true;
boolean b = obb;

Integer intObj1 = 23; //autoboxing
int intPrimitive = intObj1; //unboxing
intPrimitive++;
Integer intObj2 = intPrimitive; //autoboxing
```

Exemplu scriere clasa Complex

```
public class Complex {
      private double a, b;
      ...
      public Complex aduna(Complex comp) {
            return new Complex(a + comp.a, b + comp.b);
      }
      public boolean equals(Object obj) {
            if (obj == null) return false;
            if (!(obj instanceof Complex)) return false;
            Complex comp = (Complex) obj;
            return ( comp.a==a && comp.b==b);
      }
      public String toString() {
            if (b > 0) return a + ”+” + b + ”*i”;
             return a + “” + b + ” *i";
      }
}
...
Complex c1 = new Complex(1,2);
Complex c2 = new Complex(2,3);
System.out.println(c1.aduna(c2));            // 3.0 + 5.0i
System.out.println(c1.equals(c2));            // false

OBS: System.out.println(c1);       //apeleaza implicit toString() !!
```